

Homework 2: Scheduling Simulator

For assignment-related questions please refer to the relevant forum, and for personal questions only please refer to: asaliyam@bgu.ac.il

Note: This exercise may take time. Start early, Extensions will be granted only in exceptional circumstances. Good Luck!

Part 1: Coding Assignment

Introduction

In this assignment you will implement a CPU scheduling simulator in C. The simulator models the execution of a fixed set of processes under one of two scheduling algorithms.

Implementation Requirements

Your submission must follow these rules:

- Your code must implement the public interface exactly as it appears in the provided header files.
- You may add any internal helper functions or private data structures inside your own `.c` files.
- You must not change any function signatures, types, or names in the header files.
- Your code must not print anything and must not call `exit()`.
- Do **not** provide a `main()` function. During testing, we will compile your source files into a static library and link it with our own main program.

Processes

Processes are represented by process control blocks (`pcb_t`). Each process has:

- a process identifier (`pid`)
- an arrival time
- a total CPU burst length
- optional I/O behavior
- a scheduling state
- accumulated statistics

Processes move through the following states:

NEW $\rightarrow$ READY $\rightarrow$ RUNNING $\rightarrow$ BLOCKED $\rightarrow$ READY $\rightarrow \dots \rightarrow$ TERMINATED.

All fields of `pcb_t` must be correctly maintained by your scheduler.

Scheduling Algorithms

You must support exactly two scheduling algorithms.

First Come First Served (FCFS)

- Non preemptive.
- The READY process that has been waiting the longest is chosen to run.
- If two processes have been READY for the same amount of time, the process with the smaller `pid` is chosen.
- Once selected, the process runs until it blocks for I/O or finishes.

Round Robin (RR)

- Preemptive.
- Each RUNNING process receives a fixed time quantum (a positive integer).
- If the process is still RUNNING when its quantum expires, it moves to READY and the next READY process is chosen.
- The choice among READY processes is done in cyclic order, as in a FIFO queue that is traversed with a circular pointer.

The choice of algorithm for a simulation run is indicated by the `algorithm.t` value passed to `scheduler_create`.

I/O Blocking

A process may be I/O bound. The following fields in `pcb_t` control this behavior:

- `is_io_bound`
- `io_frequency`
- `io_duration`

If `is_io_bound` is nonzero and `io_frequency` is positive, then every time the process has executed `io_frequency` units of CPU since its last I/O, it must:

- change its state from RUNNING to BLOCKED
- stay BLOCKED for exactly `io_duration` time units
- after that move to READY

The counters `io_counter` and `io_remaining` in `pcb_t` are provided for your implementation.

Time and Simulation Model

The simulator operates in discrete time units. Each call to `scheduler_step` advances the simulation by exactly one unit of simulated time.

During each step, the scheduler must:

- update the global simulation time,
- update process states according to arrivals, CPU execution, and I/O behavior,
- manage the RUNNING process (if any),
- apply the rules of the selected scheduling algorithm (FCFS or RR),
- update all required per-process statistics.

Library Functions

You must implement all functions declared in `scheduler.h`.

`scheduler_create`

Signature:

```
scheduler_t *scheduler_create(pcb_t *processes,
                              int process_count,
                              algorithm_t alg,
                              int quantum);
```

Description: Initializes a scheduler instance. The array `processes` contains `process_count` PCBs that describe the processes to be scheduled. The `alg` parameter selects the scheduling algorithm. The `quantum` parameter is used only for RR and ignored for FCFS.

Return value: On success, returns a pointer to a scheduler object. On failure (for example, invalid parameters or memory allocation failure), returns `NULL`. This function must not print anything and must not call `exit`.

`scheduler_step`

Signature:

```
void scheduler_step(scheduler_t *sched);
```

Description: Advances the simulation by one time unit for the given scheduler, updating all process states and internal statistics.

Return value: None. If `sched` is `NULL`, the function must return immediately without any effect.

`scheduler_all_finished`

Signature:

```
int scheduler_all_finished(scheduler_t *sched);
```

Description: Checks whether all processes managed by the scheduler have reached the `TERMINATED` state.

Return value: Returns 1 if all processes are finished. Returns 0 otherwise. If `sched` is `NULL`, returns 1.

`scheduler_get_metrics`

Signature:

```
void scheduler_get_metrics(scheduler_t *sched,
                          double *avg_wait,
                          double *avg_turnaround,
                          double *avg_response,
                          double *cpu_util,
                          double *throughput,
                          double *fairness);
```

Description: Computes the final scheduling metrics based on the information collected during the simulation. The function may write the following values, depending on which output pointers are non-NULL:

- **Average waiting time**
- **Average turnaround time**
- **Average response time**
- **CPU utilization**
- **Throughput**
- **Fairness** (measured by maximum waiting time minus minimum waiting time among all processes)

If a pointer is NULL, the corresponding metric is skipped. If sched is NULL, the function returns immediately without writing to any output pointer.

scheduler_destroy

Signature:

```
void scheduler_destroy(scheduler_t *sched);
```

Description: Releases all memory and resources associated with the scheduler object. Does not free the PCB array, which is owned by the caller. If sched is NULL, the function returns immediately.

Submission

In this exercise only, no main function should be included. The Makefile must generate only the static library `libscheduler.a`, without creating any executable.

Submit the following files:

1. `process.h`
2. `scheduler.h`
3. `scheduler.c`
4. Makefile that builds the static library `libscheduler.a`

The provided `demo.c` illustrates how your library will be invoked during testing. It is your responsibility to maintain correct values in all struct fields during the simulation, since these fields will be used to evaluate your implementation.

Part 2: Theoretical Questions

1. Consider a system with a single processor and a non preemptive scheduler. There are four independent processes. Each process has:

- a weight w_i that represents its priority order, and
- a runtime t_i .

The scheduler always prefers to run processes in *ascending* order of weight (lower weight means higher priority). If two processes have the same weight, the one that arrived earlier (according to its arrival time) runs first.

The data are:

$$w_1 = 3, \quad w_2 = 3, \quad w_3 = 1, \quad w_4 = 4$$

$$t_1 = 5, \quad t_2 = 2, \quad t_3 = 8, \quad t_4 = 4$$

$$a_1 = 0, \quad a_2 = 1, \quad a_3 = 2, \quad a_4 = 0$$

- A. What is the execution order of the processes, taking into account the weights, arrival times and runtimes?
 - B. What is the turnaround time of each process?
 - C. How many different valid execution schedules are possible if the weights are not unique?
2. Consider an operating system with n processes, where process p_i is generated at time $t = i$ (for example, p_0 at $t = 0$, p_1 at $t = 1$, $\dots$, p_{n-1} at $t = n - 1$). Each process requires exactly $3n$ CPU cycles, and produces its first output after 3 cycles of CPU execution. Neglect the time required for performing the output.
 - A. Suggest a scheduling policy that minimizes the average response time, and justify your answer.
 - B. Is there any scheduling policy that minimizes both the average response time and the average turnaround time at the same time? Justify your answer.